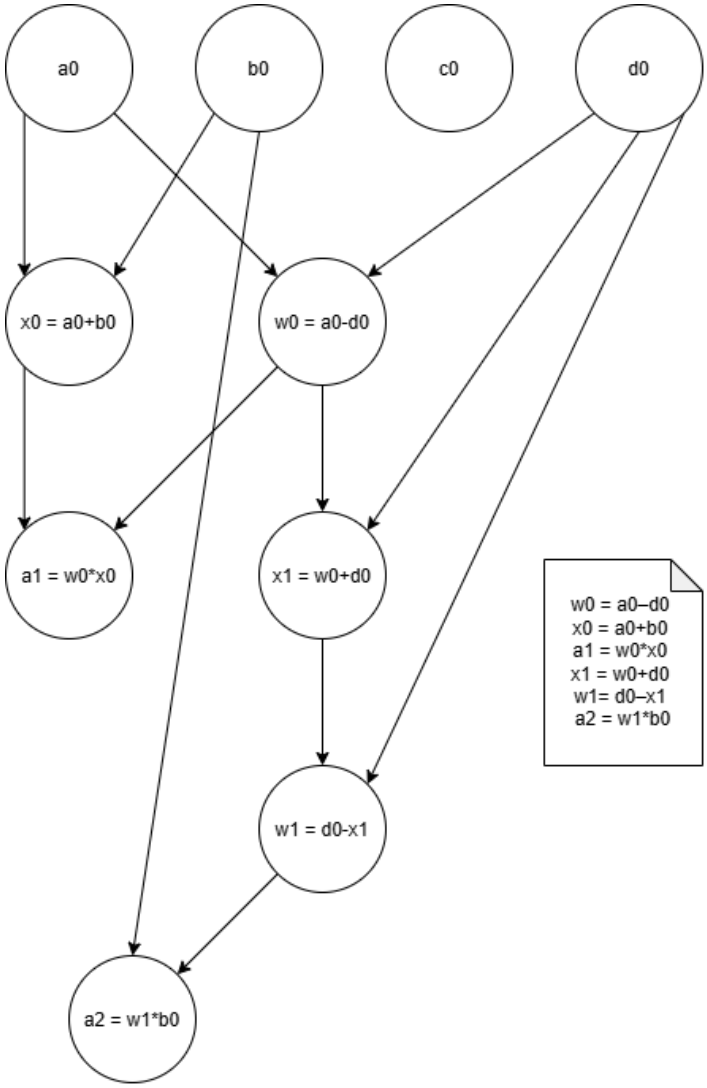
Shani “Dill” De Silva HW3 EC551

1. single assignment qs
    a. Single Assignment form of equations

        w0 = a0–d0
        x0 = a0+b0
        ra1 = w0*x0
        x1 = w0+d0
        w1= d0–x1
        a2 = w1*b0

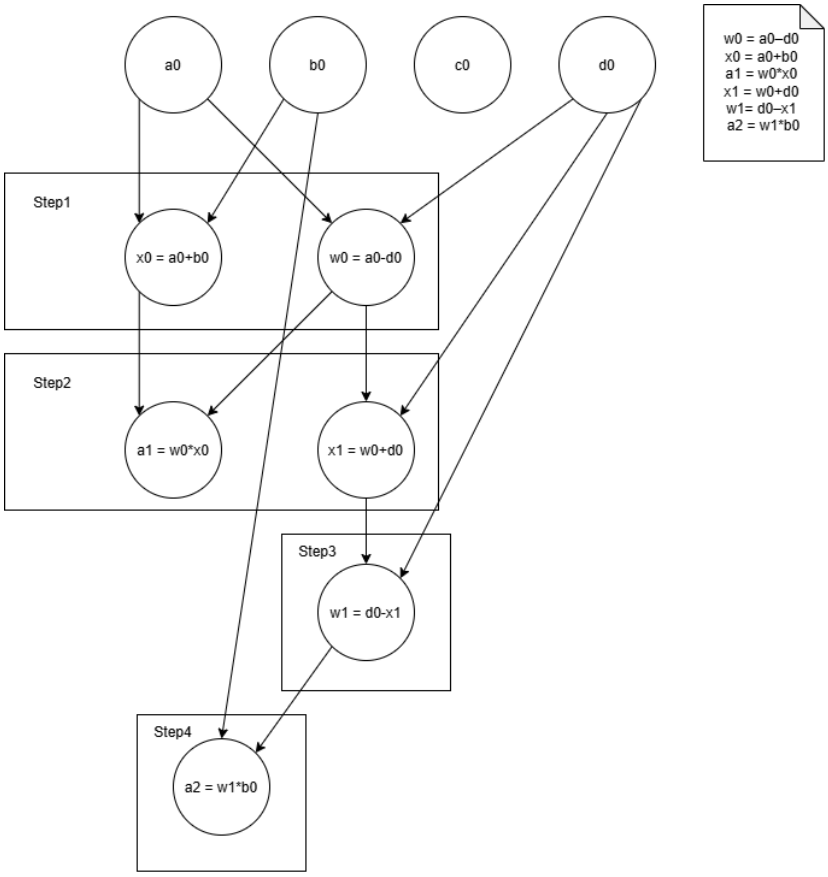
    b. Flow graph from equations in 1a



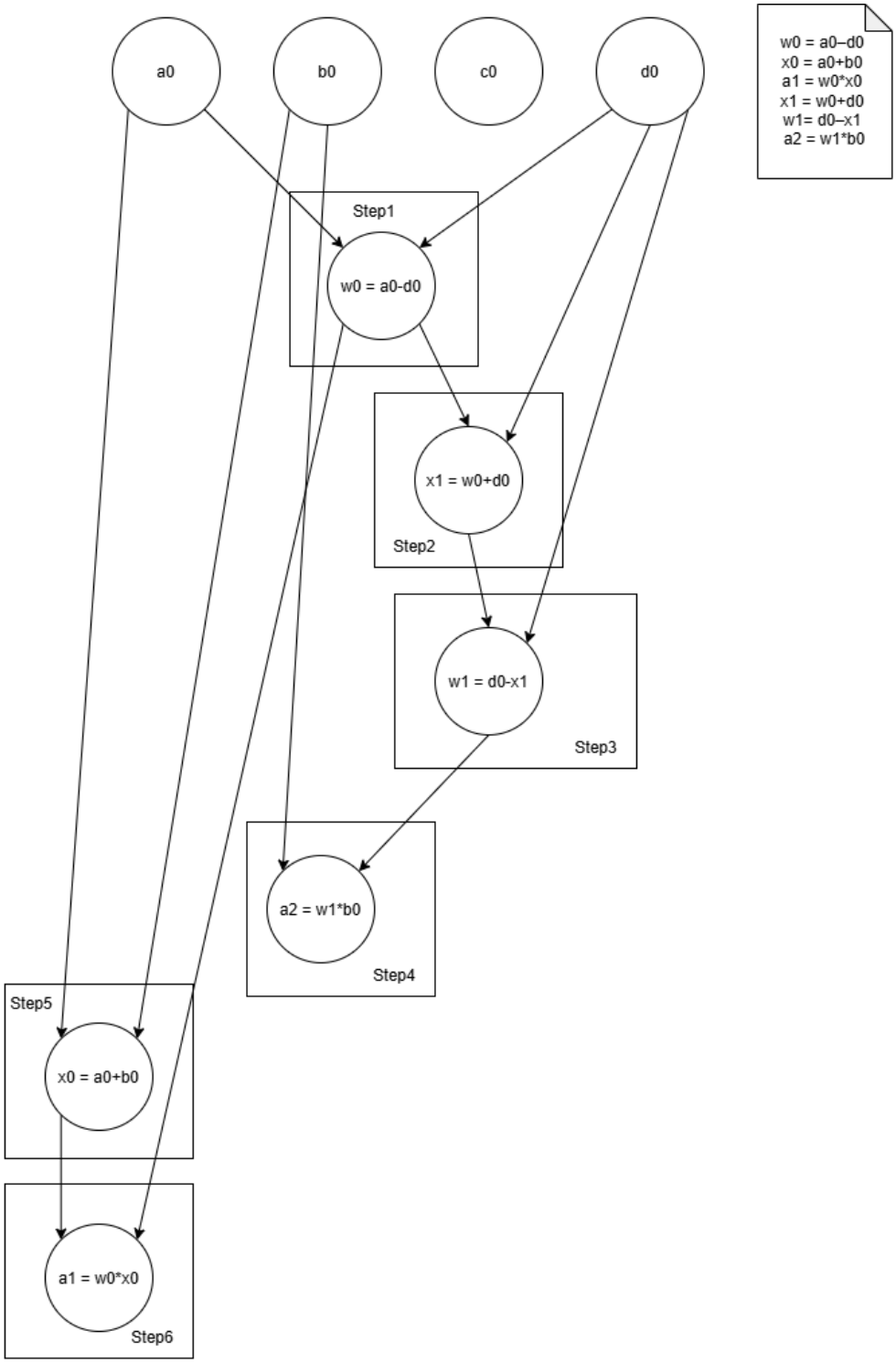
    c. Partition (and draw out) the dataflow graph into an ASAP schedule. Partition the dataflow graph into an ALAP schedule (where no cycle is idle but the design is completely sequential with one operation per cycle). Partition the graph into a “register consolidation” schedule (another form of ALAP scheduling yet

operations happen in parallel). How many total cycles are required assuming each operation takes 1 cycle?

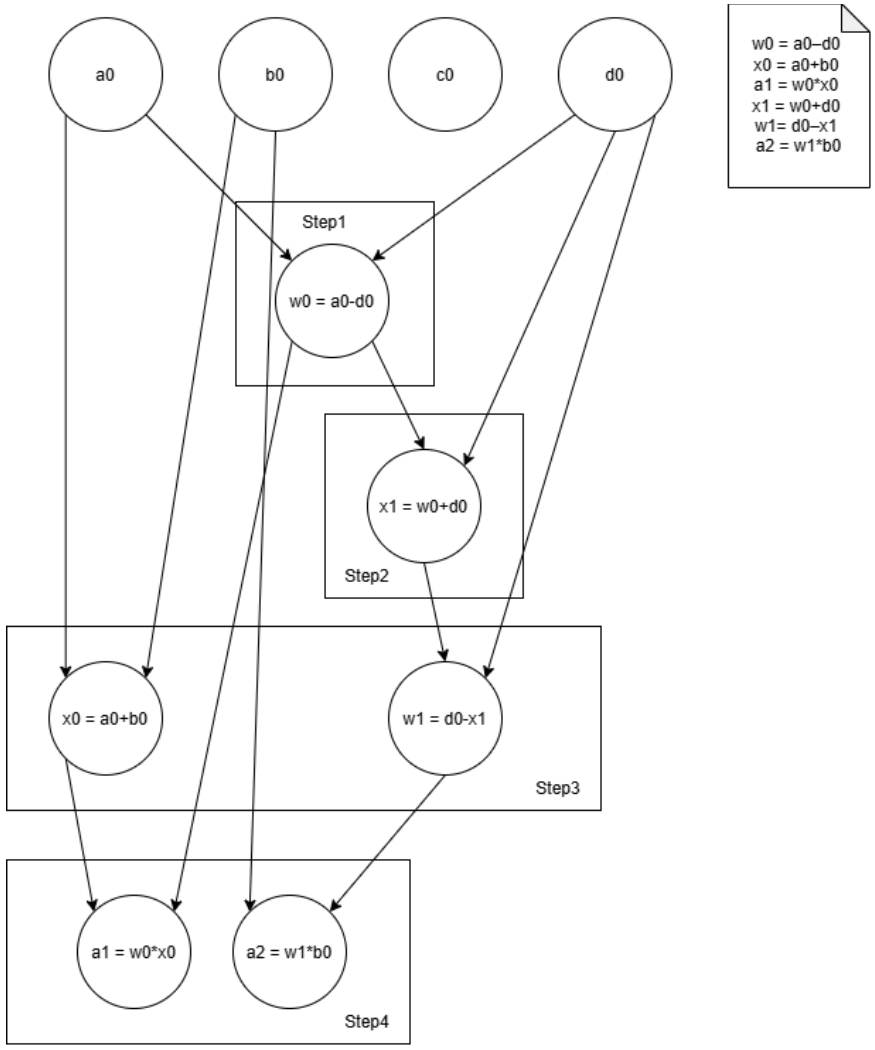
i. ASAP: 4 cycles.



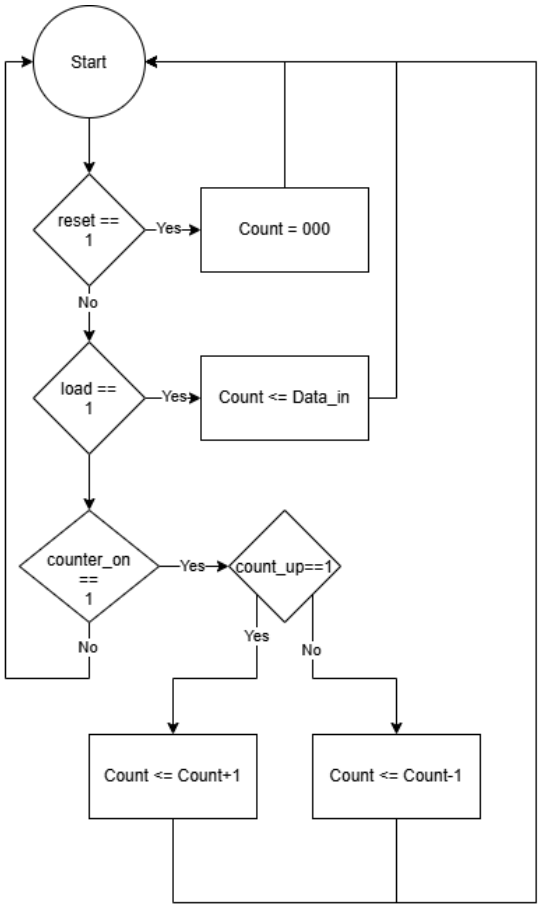
ii. ALAP: 6 cycles



iii. ALAP w. register consolidation: 4 cycles



   d. How many functional units are required for each of these schedules (e.g. how many adders, subtractors, and multipliers)?
      i. ASAP: 1 subtraction, 1 addition, 1 multiplication.
      ii. ALAP: 1 subtraction, 1 addition, 1 multiplication.
      iii. ALAP with Reg. Consolidation: 1 subtraction, 1 addition, 2 multiplication.
   e. Next steps would be to decide on scheduling, instantiate basic modules in verilog and create some initial test data. It is not reasonable to assume 1 cycle per operation, as multiplication takes significantly longer, so I would suggest scheduling as many operations as possible to take place parallel to multiplication where possible.
2. Questions from book: ASM design. only create ASM chart.
   a. 5.43: Draw an ASMD chart for the up-down counter described in Example 5.42.



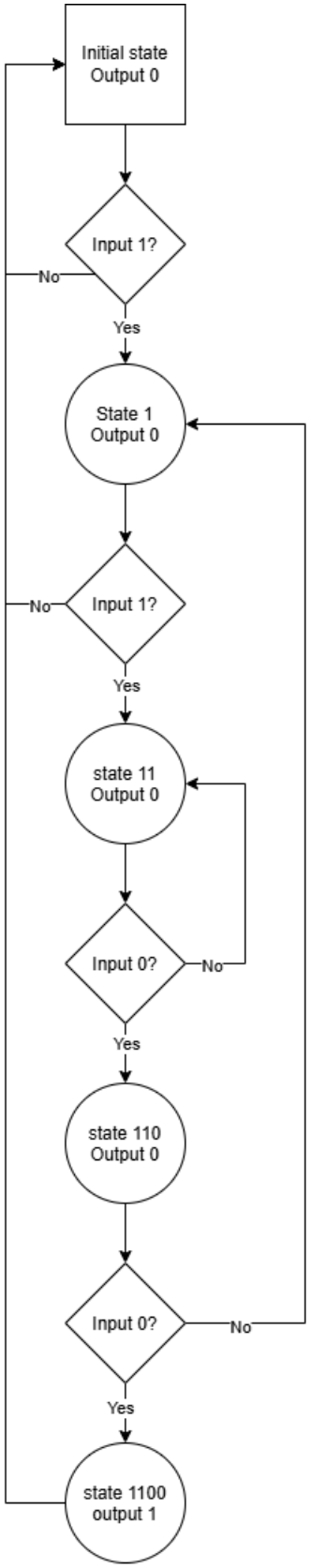
Our last example of a counter is a 3-bit up–down counter, but modified to include two additional features: a signal, counter_on, which enables the counter, and a signal, load, which loads an initial count from an external datapath.The description of the counter exploits Verilog's built-in arithmetic and implements the counter with an ifstatement. The synthesized circuit and block diagram are shown in Figure 5-29. In this implemen-tation, the library cell dffrgpqb_a is a D-type flip-flop active on the rising edge, having internally gated data32 and asynchronous active-low reset.

```
module up_down_counter (
output reg [2: 0] Count,
input input [2: 0]
);

always @ (posedge clk, posedge reset)
      if (reset == 1'b1) Count <=3'b0;
      else if (load == 1'b1) Count <= Data_in;
      else if (counter_on == 1'b1) begin
            if (count_up == 1'b1) Count <= Count+1;
            else Count <= Count-1;
      end
endmodule
```

b. 5.48: Draw an ASM chart of a Mealy machine that samples a serial bit stream on the rising edge of the clock and asserts an output if the last four samples are such that two ones are followed by two zeros.

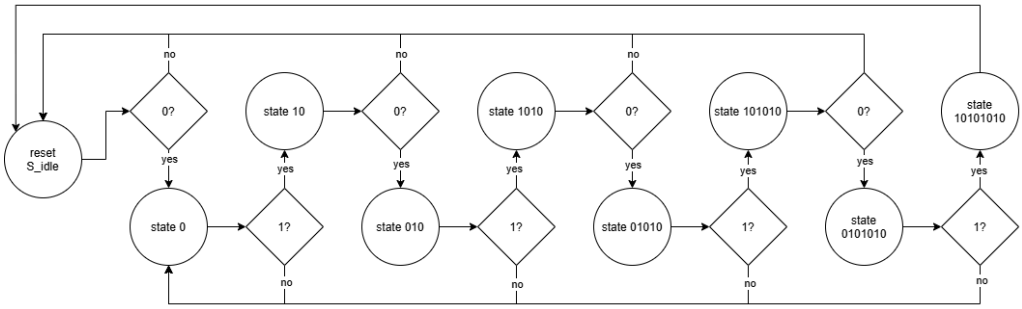
Shani “Dill” De Silva HW3 EC551



5.48: Draw an ASM chart of a Mealy machine that samples a serial bit stream on the rising edge of the clock and asserts an output if the last four samples are such that two ones are followed by two zeros.

c. 6.5

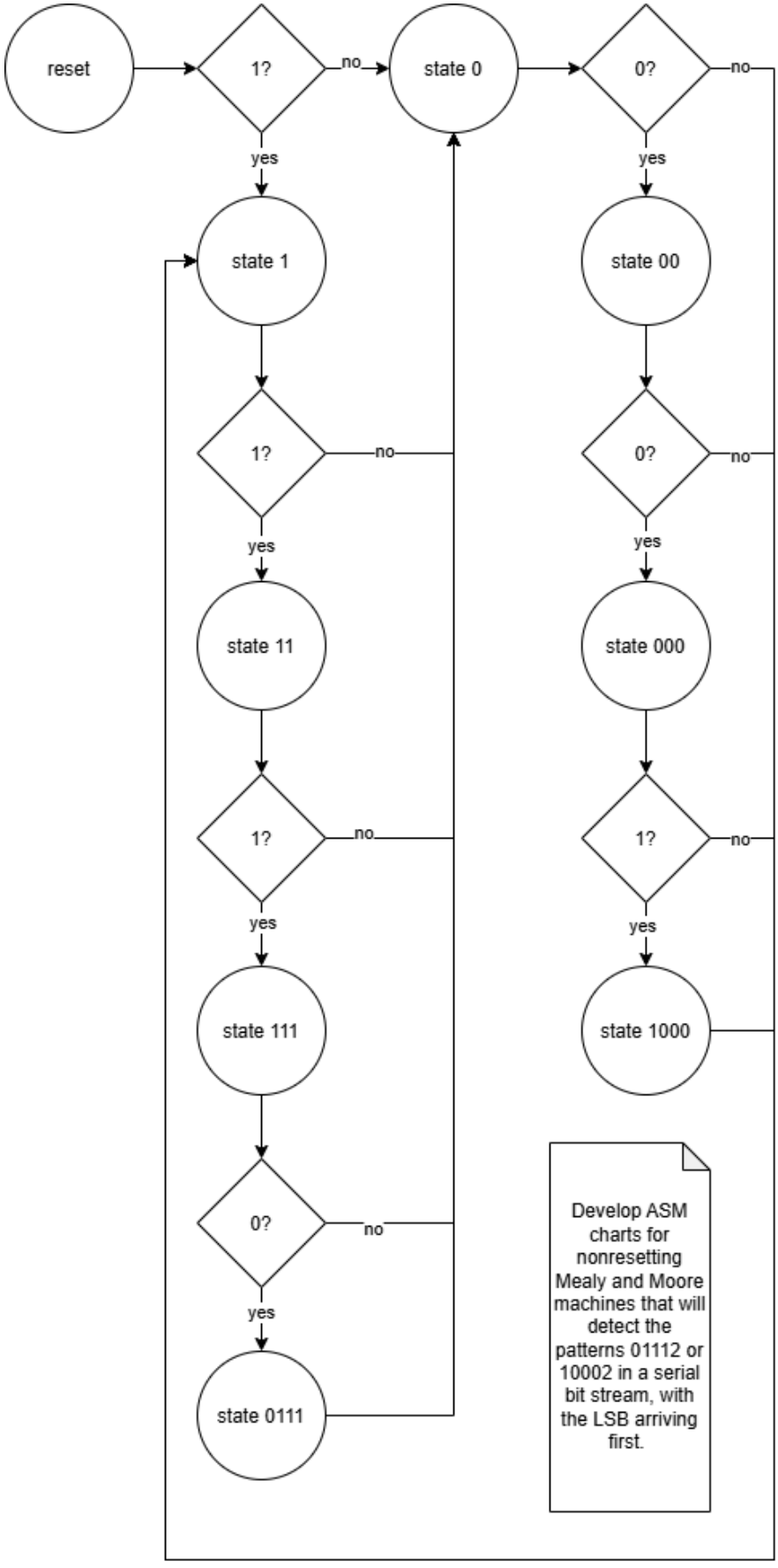
Develop ASM charts for resetting Mealy and Moore machines that will detect the pattern 10101010_2 in a serial bit stream (with the LSB arriving first), assert their output, and then return to the reset state, S_idle, where the outputs are de-asserted, before processing additional bits.



d. 6.7

Develop ASM charts for nonresetting Mealy and Moore machines that will detect the patterns 0111_2 or 1000_2 in a serial bit stream, with the LSB arriving first.

Shani “Dill” De Silva HW3 EC551



3. MIS paper reading
    a. What is a “multi-level logic expression”? Why do they exist? Why might they be advantageous over two-level logic expressions?

        It is a boolean logic expression that requires more than two levels of logic to efficiently describe it. For example, (AB+AC)(DE+FG) is certainly representable in two levels of logic, but the description would be far longer than necessary. As the paper says, “Two-level logic representations can be viewed as special cases of more general multilevel representations”. Multi-level Logic Expressions therefore have all the benefits of two-level logic expressions, as those two-level expressions are necessarily multilevel expressions, but the wider group allows for more flexibility and the possibility that “a multilevel representation can be implemented in less area and generally as a much faster circuit”.

    b. What is the purpose of this paper (In your OWN words)?

        The purpose of this paper is to improve the simplification of boolean expressions to make it easier to implement them in a circuit. It does this by a) not sticking to the standard two-level logic, which can limit implementations by flattening them, b) looking at different ways to simplify out terms with factoring, c) using two kinds of division of different speeds and efficiencies to make simplification faster, and d) finding ‘subexpressions’ that could be made more efficient.

    c. What is a “factored form”? Provide an example NOT in the paper.

        A factored form is the expanded definition equivalent to the “sum of products” form. Technically the literal Z is in factored form, and in minimum factored form, and not given in the paper, but a more fun factored form example would be:

        wxyz+wxz+yw simplifying to w(xz+y)

    d. What are “Algebraic and Boolean division”? Provide examples of each that are NOT in the paper.

        Algebraic division treats factoring out parts of the expression like numerical division, as far as I can tell. It doesn’t need any boolean operations to arrive at the result. An example would be XY+XZ+XYW+B simplifying to X(Y+Z+YW)+B, where (Y+Z+YW) is the quotient and B is the remainder.

        Boolean division is the same as Algebraic division, but it allows for boolean expressions as well. For example, it recognises when one term in the expression would be consumed by another. In the previous example, XY+XZ+XYW+B, the resulting expression would simplify into X(Y+Z+YW)+B, because we could

recognise that YW + Y = Y.

e. What is a “kernel and a co-kernel”? What does the level of a kernel denote? Give examples of kernels and co-kernels. Specify their level as well. These should NOT be in the paper.

The kernels of an expression are the expression’s cube-free primary algebraic divisors. This means that (I’m not sure about this) the kernel is what results when the function is algebraically divided by a co-kernel, resulting in kernel-co-kernel pairs. The resulting kernel must be unable to be divided by any other co-kernels, which puts me in mind of prime factorization.

A kernel’s level represents how many sub-kernels can be used to create it, other than itself. A level-0 kernel would be a literal, like B, and a level 2 kernel would be AB+AC with the kernels A and (B+C).

An example of a co-kernel for the function abc+abd+abe could be ab, which would result in c+d+e as the kernel - however, b could not be a kernel as the function would also be entirely divisible by the literal ‘a’.